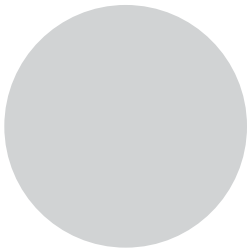


1

FOUNDATIONS OF RED TEAMING



Chess professionals often practice by playing against themselves, a practice commonly known as solo chess or game analysis.

This practice allows players to test strategies through an opponent's perspective and discover flaws and weaknesses in their strategies before trying them against a real opponent. Red teaming applies a similar approach to cybersecurity; it lets an organization test their security controls as if they were the adversary to spot weaknesses and shortcomings before real attackers do.

This chapter explains what red teaming is and how it differs from other testing practices in cybersecurity. In the process, we outline several principles about how successful red teamers think about their work, and introduce game theory concepts that are relevant in most real-world attacker/defender dynamics.

Defining Red Teaming

Red teaming, put simply, is full-scope objective-based adversary emulation.

Many offensive security approaches, such as penetration testing, start by defining a target scope (perhaps a network range, application, or specific environment). Because red team operations are *objective-based*, they begin by defining attack goals, sometimes called “trophies,” that match the objectives of real adversaries. Those goals might include gaining physical access to a secure data center or gaining access to sensitive information. These attack trophies guide every part of the operation, including which techniques the red team will use throughout the attack scenario.

Adversary emulation refers to how the red team mimics the tactics, techniques, and procedures (TTPs) of attackers. *Tactics* are high-level attack behaviors, such as initial access or lateral movement. *Techniques* provide a more detailed description of the behavior, including specific methods like phishing or exploiting public-facing applications. *Procedures* describe those techniques in detail, even including the specific tooling and command syntax an actor uses. A red team that emulates a specific adversary studies their TTPs, just as a chess master analyzes an opponent’s patterns, to test defenders against attack scenarios that are accurate and realistic.

Adversary emulation aims to replay a specific, known actor’s TTPs as faithfully as possible. It focuses on what the threat actor has done previously, based on available threat intelligence, and closely adhering to those same approaches. Its close relative, *adversary simulation*, drops these rigid restrictions to demonstrate what a similar threat actor could do, allowing testers to apply greater creativity to adapt attack activity to the nuances of their specific environment. Realistically, no emulation can be perfect, since intelligence is always incomplete and attackers typically change their TTPs over time. Skilled red teams apply a balance between these approaches, by staying faithful to an adversary’s overall intent and capabilities while adapting techniques to the target environment. Keep in mind that the industry often uses these terms interchangeably and sometimes inconsistently. We’ll use emulation generally throughout this book and make specific distinctions wherever they are needed.

Let’s consider a red team operation modeled on Burly Vortex, a fictional ransomware group we’ll be using throughout this book to represent a financially motivated threat actor. A pure emulation of Burly Vortex would replicate their past campaigns step by step, perhaps even using the exact same phishing templates and malware payloads the group had used in past breach activity. A simulation, on the other hand, might use Burly Vortex’s attack playbook as a general reference, but would allow flexibility to adapt any TTPs when the target environment calls for it. For example, the red team might modify their approach for gaining initial access, if the original path is not available, or apply different lateral movement techniques that serve a similar purpose. Both simulation and emulation have unique benefits for training defenders and validating defenses.

Red team operations are also full-scope by default. *Scope* is a testing engagement's set of boundaries. Most security testing operates within tightly drawn scopes to manage risk or focus on specific concerns. Real-world adversaries do not respect scope boundaries. They target any combination of people, facilities, systems, and third parties that accomplishes their goal. A red team mimics that behavior as closely as possible within legal, ethical, and safety constraints. In later chapters, we'll cover how to plan your operations in a way that maximizes results while minimizing the potential risks involved with emulating attackers.

An Example Red Team Operation

Imagine that the security team at ACME Corp, a major aerospace component manufacturer, has been tracking an increasing trend of ransomware attacks targeting the aerospace sector. Leadership has grown concerned about exposures to both the operational technology that runs its manufacturing floors and the third-party providers that transport components between ACME's facilities. The security team and its stakeholders request a red team operation to test how the organization would hold up against such an attack scenario.

A small, trusted insider group kicks off the initial planning process. The chief information security officer (CISO), a couple of senior security leaders, stakeholders from affected business units and a representative from Legal hold a meeting to discuss the operation. This team chooses *Burly Vortex* as the threat actor on which to base the operation, and sets three high-level attack goals similar to ones this group would actually pursue: gaining access to confidential design schematics, altering shipment schedules in the logistics systems, and establishing persistent access to engineering networks. The group documents formal rules of engagement that outline the activities the red team is authorized to perform and who to contact if escalations are required.

The operation runs for six weeks. The red team starts by performing reconnaissance and targeted social engineering against members of the operations and engineering teams, yielding an initial foothold into ACME's internal network. The team then moves laterally and escalates privileges, as they move closer toward reaching the three attack trophies previously defined. When an attempted initial access method fails partway through the operation (because a vigilant defender sees it and responds), the team retries and adapts its approach. The red team keeps the trusted insider group informed throughout, but to maintain realism, does not provide any advance warning to the blue team, including security operations center (SOC) analysts and incident responders. After all, the whole point of the exercise is to see how the organization responds to an unexpected attack scenario.

After the operation concludes, the red team holds a series of outbrief sessions with different stakeholder audiences. The red team first gives the SOC team a detailed technical walkthrough of the attack activities to understand whether detective controls worked as intended (or didn't). The

CISO and executive stakeholders get a higher-level narrative about which attack objectives the red team achieved, which scenarios were blocked by defenders, and any critical or thematic issues observed around ACME's resilience against a real Burly Vortex campaign. Lastly, the red team shares with engineering leadership several weaknesses they observed in the logistics systems, offering suggestions to remediate the gaps.

Red Teaming vs. Vulnerability Scanning vs. Penetration Testing

Red teaming is just one of many forms of offensive security testing, and it isn't always the right tool for a given job. When should you choose red teaming instead of vulnerability scanning, penetration testing, or other offensive security approaches?

Vulnerability scanning is an automated process used to identify known vulnerabilities and misconfigurations across systems, networks, and applications. A scanner or endpoint agent examines operating systems, open ports, running services, and configurations, and compares the results against a database of known issues. These scans can run continuously with minimal overhead and even integrate seamlessly with ticketing systems to triage findings to system owners. As such, vulnerability scanning is a foundational element of most organizations' ongoing security hygiene. This automation does, however, come with some drawbacks. Scanners tend to generate high volumes of findings, including many false positives. They also rarely identify exploitable chains that involve multiple steps against different services. Humans typically need to validate true positives via manual efforts (although AI is beginning to reduce this dependency significantly).

Penetration testing, by comparison, applies some level of automation but relies heavily on creativity and judgment to chain multiple vulnerabilities together and illustrate what an attacker could accomplish with the findings discovered. Given this additional level of analysis and decision-making required, it's more expensive than scanning and produces far fewer findings, but those it does present are typically much more accurate and contextualized. Penetration tests usually operate within a predefined scope that is technical-only, which leaves other aspects of an attack surface (like humans, physical controls, process gaps, and organizational blind spots) untested.

Red teaming's scenario-driven, full-scope approach is the most expensive and time-consuming of the three methodologies, but it's also uniquely able to determine how well an organization could hold up against a motivated, capable adversary.

To illustrate the differences, consider how each approach might assess a bank's SWIFT environment. (SWIFT is the global messaging network banks use to authorize interbank transfers, which makes it a target for financially motivated attackers.) A vulnerability scan would quickly and efficiently identify known weaknesses in the underlying systems, producing a list of common vulnerabilities and exposures (CVE) IDs and risk scores. A penetration test would find exploitable vulnerabilities and demonstrate how an attacker might chain them together to compromise the SWIFT infrastructure.

A red team operation would start from the attacker's goal of moving money out of the bank, unauthorized and without detection. The team might begin by performing targeted phishing and vishing of payment operations staff, then pivot over the internal network to access the SWIFT environment, exploit chained technical weaknesses along the way, and test whether the bank's detection and response teams catch the malicious activity. Its findings would span people, processes, and technology, and they would reveal whether the bank's end-to-end defenses can withstand a realistic attack.

Learning from Game Theory

Game theory is a branch of mathematics focused on the study of strategic interactions among rational decision-makers. It gives us formal ways to reason about decisions that depend on what other actors do and about the outcomes that result when each side chooses its moves based on what it believes the other sides know and want. For red teamers, game theory is most useful as an analytical lens. Cybersecurity is fundamentally a domain of strategic interaction between attackers and defenders, and many of the dynamics red teams encounter in the field have been studied formally for decades.

Four game theory concepts are particularly relevant to red teamers: zero-sum games, which describe scenarios where one side's gain is the other side's loss; information asymmetry, where one side knows things the other doesn't; signaling games, where actors convey information to influence the other side's decisions; and bounded rationality, which recognizes that real decision-makers work under constraints and can't evaluate every option. Each of these concepts gives red teamers a sharper way to think about how attacks actually unfold and why defenses succeed or fail.

Zero-Sum Games

Cybersecurity contains plenty of genuinely *zero-sum* moments. When an adversary exfiltrates data or disrupts operations, the attacker gains what the organization loses. Red team engagements, however, are not zero-sum. Although the red team and the blue team are notionally on opposing sides of an exercise, they work toward the same goal of strengthening their organization's security program. A red team's success at compromising a critical system undetected is a win for the blue team, which gains information about which parts of the organization's detection and response process need work.

Red teams, too, benefit from running their simulations against capable, well-instrumented defenders. Each engagement sharpens their understanding of how modern detection stacks behave and how response playbooks execute under pressure. A well-run red team program generates results that neither side could achieve alone.

Information Asymmetry

Information asymmetry, which describes situations in which one party has more or better information than another, shapes attacker-defender dynamics more than any other game theory concept in this chapter.

The Battle of Midway during World War II shows the power of information asymmetry. On the morning of June 4, 1942, Japanese Admiral Chūichi Nagumo had launched a first wave of aircraft against Midway Atoll, a strategically vital American outpost in the central Pacific. The strike damaged facilities across the island, but the wave's commander radioed back that a second attack would be needed. Nagumo had held a second wave of aircraft in reserve on his carriers' hangar decks, armed with anti-ship weapons in case American carriers appeared. With no American ships yet sighted, he ordered the reserve aircraft re-armed with land-attack bombs to finish the job on Midway. In the middle of re-arming, a scout plane reported American ships to the northeast, including a carrier.

This situation became known as Nagumo's Dilemma. He could halt the re-arming and restore anti-ship weapons on the reserve aircraft for an immediate launch. The strike would go out piecemeal, without fighter escort, and the first-wave aircraft now returning low on fuel would have nowhere to land. Alternatively, he could recover the first wave of returning planes, re-arm everything for a coordinated anti-ship strike, and launch roughly 45 minutes later. Nagumo chose the coordinated option, as Japanese naval doctrine encouraged. While his flight decks were full of fueled aircraft and ordnance staged from the re-arming reversal, American dive bombers found him. Three of his four carriers were fatally hit within minutes, and the fourth fell that afternoon. The Imperial Navy never recovered its offensive footing in the Pacific.

The Americans had been reading Japanese naval communications for weeks. They knew Midway was the target, they knew Nagumo's approximate position and timetable, and they had positioned three carriers in ambush before he ever launched his first strike. Nagumo was making consequential decisions under uncertainty about whether American carriers even existed in the area.

In cybersecurity scenarios, you may assume that the attacker holds the information advantage. After all, they choose when to strike, which techniques to use, and what objectives to pursue, while the defender has to prepare for a wide range of possibilities. However, within mature security organizations, defenders gain significant advantages of their own that can give them an upper hand in attack scenarios. Defenders know their environment, including which systems exist and what traffic flows and processes are normal and abnormal.

Claude Shannon described this dynamic in what's now called Shannon's Maxim, which states that systems should be built under the assumption that the adversary understands their general design. Details about systems can leak in many ways that defenders can never fully control, including through ill-advised human decisions. Security that relies on design details remaining hidden can fall apart the moment they are exposed. Assuming the

adversary already knows your design forces you to build that system in a way that is fundamentally sound, no matter what information attackers have.

But in practice, even if attackers usually understand the general patterns of common security products, such as a well-known EDR product, they won't necessarily know the nuances of the configuration in front of them, including any custom alerting the organization may have built based on unique business logic. This is one of many examples of how defenders can gain distinct advantages over attackers.

AI AND THE ATTACKER'S LEARNING CURVE

A recent dynamic that is important to watch carefully is how AI tools are changing a defender's advantage in knowing their network. Historically, attackers had to invest considerable time learning about the target environment, reading through internal documentation, mapping systems, and building a mental model of the business before performing further attack activity. AI-assisted analysis is compressing that timeline significantly. Today, an attacker with access to internal wikis, ticket systems, or code repositories can use language models to synthesize large volumes of organizational knowledge quickly and can ask targeted questions to learn where sensitive systems are or how specific business processes work. While the defender's information advantage still exists, AI is enabling attackers to close the gap. Red teams should increasingly model and test this dynamic.

Well-designed red team engagements help defenders test and strengthen their asymmetry advantages. An engagement can serve to validate whether a large defender advantage exists or whether it's only nominal.

Signaling Games

Signaling games describe interactions in which one party conveys information, whether truthfully or deceptively, to influence another party's decisions. Signaling is relevant to attacker-defender dynamics because the conclusions each side draws about the other depend heavily on the signals they observe. An attacker who can manipulate the signals a defender relies on gains a powerful operational advantage; a defender who can recognize manipulated signals, or plant signals that mislead an attacker, gains the same.

Consider a campaign one of our teams once ran. We created a fictional recruiter persona on LinkedIn, purportedly looking to fill roles at the target company. Over the course of three weeks, we built that persona's network, connecting with hundreds of people in the organization, and otherwise behaved as a normal recruiter would. After the persona had established itself, we began directly reaching out to specific employees at the target company with links to job postings hosted on its official website. The first message we sent included a link to a real opportunity. So did the second,

which we sent a week later. By the third message, the recipient had determined this recruiter was a known, trustworthy contact who sent relevant roles. That third link went to a posting on infrastructure we controlled and required the recipient to take a small action to view it. The recipient completed the action, which executed our payload.

Every signal we sent in the early phase was authentic. The deception lay in the channel itself, not in the messages. The early authentic content was an investment in credibility, ensuring that a single deceptive signal at the end would land without raising suspicion.

Defenders use similar techniques, such as *honeypots* and *deception networks*: signaling exercises in which defenders construct systems that present themselves as legitimate, valuable production assets to anyone reconnoitering the environment. These systems incorporate signals like realistic naming conventions and believable service banners. When attackers act on those signals, they reveal their TTPs while gaining nothing operationally useful in return.

A red team engagement might evaluate whether anomalous traffic patterns get flagged or dismissed, or whether a slow drip of low-severity alerts reaches the threshold necessary for someone to investigate further. Adversary emulation that blends into expected baseline activity is particularly valuable because it closely replicates real attacks.

Bounded Rationality

The economist and cognitive scientist Herbert Simon coined the term *bounded rationality* in the 1950s to describe how decisions actually get made. Decision-makers work with limited time and information, and they choose among the options they can identify rather than searching exhaustively. Nagumo's Dilemma exemplifies this concept well, as Nagumo had to commit to a choice under intense time pressure and with limited information, knowing his decision would be irreversible once it was made.

Bounded rationality has direct applications in red teaming. Engagements should always be grounded in what real adversaries are likely to do given the constraints they face, instead of what they theoretically could do with infinite time and budget. An attack path that requires a year of dedicated reconnaissance or an exorbitant amount of AI token cost is unlikely to model the true threat the organization faces. A useful red team operation tests against what is feasible and probable for adversaries to do.

Defenders can also use bounded rationality to their advantage. Adding custom detection logic and additional depth of countermeasures can raise the cost and feasibility of an attack. When costs climb high enough, many attackers will abandon the effort to pursue softer targets.

A Red Team's Mindset

Red teaming is a craft, and like any craft, it depends as much on how the practitioner thinks as on what they know. Operations almost never go completely according to plan. Operators hit snags when compiling payloads or

maintaining persistent access. The greatest challenge for red teamers is often staying focused when things go wrong. The following principles describe the mental models that lead great operators to produce top-quality work.

Principle 1: Do No Harm

In red teaming, “do no harm” starts with the obvious. Don’t crash production systems. Don’t destroy data or make irreversible changes. At a deeper level, it also means never inadvertently degrading the organization’s security posture or burning stakeholder trust.

Suppose you’ve just gained access to a database that contains sensitive HR information. It would be quick and easy to just dump the entire table to provide irrefutable proof of your access. But should you? Providing a row count, plus a single redacted row, could demonstrate the same level of access without the risk of exposing the full dataset. As a rule of thumb, always take on the minimum level of risk required to make your point, even if it means performing a few extra steps.

Doing no harm also means working with legal, compliance, and HR partners throughout initial planning stages, especially when an operation will touch sensitive areas of the organization. Treating those partners as collaborators ultimately results in safer and faster operations. In the long term, a collaborative, disciplined approach builds trust, allowing red teams to continue doing interesting and consequential work.

Principle 2: Trust but Verify

One of the most valuable traits in red team tradecraft is healthy skepticism. Organizations operate with a long list of unstated assumptions about how their controls are configured and their environments are protected. Many of those assumptions don’t hold up under scrutiny. Red teams are well-positioned to find the gaps between an organization’s perception and its reality.

Say a security organization claims to have universal multi-factor authentication coverage. In practice, certain legacy applications may predate the coverage’s rollout, or forgotten service accounts may never have been enrolled. The same pattern commonly shows up in cloud security. Leadership might assert that secure-by-default policies protect every cloud asset, but a red team often finds publicly exposed storage buckets and over-permissioned roles that prove the assertion incomplete.

The principle applies to the team’s own work, too. Just because a technique succeeded in a previous engagement doesn’t mean it will work again, and just because an attack path looks viable doesn’t mean it’s the best one. Red teams should pressure-test their own assumptions just as carefully as they check the organization’s to produce reliable findings and model the evidence-based intellectual rigor they want the rest of the organization to practice.

Principle 3: Embrace Uncertainty and Complexity

Red team operations rarely follow a clean, predictable script. The environments we test are shaped by years of questionable architectural decisions, layered acquisitions, inconsistent documentation, and sometimes organic weirdness. To work in that kind of terrain, red teamers must be adaptable and comfortable making decisions when they can't fully know the situation.

Consider a common scenario: A red team plans to deliver a phishing payload via a link to a file on a common cloud storage platform (such as Google Drive). The tactic has worked in past engagements. This time, though, the target organization has URL filtering in place that blocks outbound access to that platform, and the link gets stripped before it reaches the recipient. The original plan won't work. The team has to decide what to do next and needs to make that decision quickly enough that the operation doesn't stall. Maybe the answer is to pursue a different delivery mechanism or another initial access path altogether. The judgment call must happen with whatever information the team has at that moment.

Such pivots are commonplace during red teaming. Being attached to a specific plan, or assuming the environment will behave like the last one did, can derail an operation. Red teams that have internalized this principle will plan every engagement thoroughly but hold the plan loosely. They expect things to go sideways and adapt, continually moving toward the mission's goal even when obstacles force path changes.

Principle 4: Stay on Target

Red team operations are full of rabbit holes and time sinks. A misbehaving payload or a troublesome security control can pull an operator's attention away from what is most important. It's easy to get absorbed in solving an immediate problem and lose track of an engagement's broader purpose.

The strongest red teams operate with dual awareness. They zoom in to handle the technical detail in front of them, then zoom out to make sure that detail is actually contributing to the engagement's objectives. No matter how clever the exploit, it can be a real distraction if it doesn't move the operation toward its goal.

Staying mission-focused requires periodic self-examination during an operation. Is the current attack path still aligned with the engagement's objectives? Would alternative approaches provide a better return for the time invested? Is the current technique the right fit or just the most familiar one? An effective red team knows when to push deeper into a path and when to step back and reassess.

Principle 5: Challenge Your Cognitive Biases

Red teaming is as much a mental game as a technical one. Human thinking isn't immune to error, and the same shortcuts that help us move fast in everyday life can quietly lead us off course in complex operations. In red team work, where stakes are high and information is incomplete, the shortcuts we don't notice tend to be the ones that hurt us the most.

A few cognitive biases come up regularly in red team operations:

Confirmation bias We tend to seek out information that supports what we already believe. A red teamer convinced that a target is vulnerable to Kerberoasting may concentrate their effort there and ignore evidence to the contrary. Strong teams test multiple hypotheses, especially when an early one seems too convenient. As AI testing has entered red teams' work, this trap has become especially common. While a web application generally returns the same response to the same request, large language models are non-deterministic and may respond differently each time, so a prompt technique that fails on the first attempt can succeed on a later one.

Anchoring bias First impressions can disproportionately shape our later interpretations. If a command-and-control beacon gets flagged early in an operation, it's easy to conclude that the defenders are highly capable and shift into overly stealthy techniques that waste time. The early information may not reflect the broader picture, so always consider later evidence on its own merits.

Groupthink When consensus forms too quickly, it can stifle dissenting ideas. Teams that explicitly invite constructive pushback will produce better ideas and avoid operational dead ends that members may otherwise have been afraid to question.

Dunning-Kruger effect People with limited knowledge or skill in a domain tend to overestimate their own competence because they lack the experience to recognize what they don't yet know. Early in a red teamer's career, a few successful operations can produce a misleading sense of mastery, leading to risky decisions and skipped precautions. The opposite tendency also shows up at the senior end of the spectrum, where experienced operators sometimes don't realize how much of what they know isn't obvious to others. Awareness of this effect helps in both directions.

Sunk cost fallacy People often keep investing in an unproductive path because of the time they already spent on it. Teams that allocate a fixed amount of time for exploring each attack path and review their progress regularly are better positioned to abandon dead ends and redirect their effort toward something more productive.

It's unrealistic to think that we can eliminate bias from our thinking. We can, however, anticipate situations in which biases may surface and apply processes around those situations to keep our assumptions in check.

Principle 6: Know the Adversary

Adversary emulation requires getting into the mindset of a real threat actor and making decisions the way they would. The better the red team understands the adversary, the more credible their operation becomes.

The closest analog is method acting. A great actor goes above and beyond delivering lines by studying how the character thinks, what motivates them, how they speak, and how they move. For red teamers, a useful structure for

that study is these Five W's, which we've adapted from journalism and method acting:

- *Who* are you emulating? What is their skill level and funding? How do they typically behave?
- *What* do they want? Access, disruption, data theft, or something else?
- *Why* do they pursue those objectives? Financial gain, espionage, ideology, or another motive?
- *Where* do they operate? Which parts of the target organization are most relevant to their typical targeting?
- *When* would they strike? Are there time-sensitive pressures, operational constraints, or moments when the target is most vulnerable?

A financially motivated ransomware affiliate, for example, cares about quickly encrypting data and creating leverage for payment, which means the red team should prioritize initial access, fast lateral movement, and reaching encryptable data quickly. A nation-state actor pursuing long-term access cares about staying undetected for months or years, which means the red team should emphasize operational security, patient reconnaissance, and persistence mechanisms that survive defensive turnover. Different adversaries change the shape of operations.

Principle 7: Embrace Teamwork

Red teaming is collaborative by nature. The work spans social engineering, physical access, cloud abuse, lateral movement, malware development, and many other domains. No single practitioner is likely to master all of these skills.

Effective red teams amplify each other's strengths. When a cloud specialist works alongside a phishing operator, or when a physical tester hands off a foothold to an infrastructure specialist, the engagement becomes capable of attack paths that an individual may not have found alone. For example, a physical tester could infiltrate a secured office space and plant a device on the internal network, and a teammate could then use that foothold to deploy malware that exfiltrates data through a cloud channel.

Teamwork can also make an operation safer. It's easy to misfire commands and misconfigure payloads when working under pressure. Having a second set of eyes, whether through screen sharing, peer review, or a quick sanity check before hitting ENTER, prevents small mistakes from becoming outages or burned engagements.

Collaboration should extend beyond the red team itself. Red teams that build constructive working relationships with the blue team and the broader security organization produce better engagements. An overly competitive relationship with other teams can undermine growth and progress.

Principle 8: Understand the Defense

To emulate an adversary credibly, the red team has to understand the environment it's operating in and the defenses inside it. For example, a red teamer who understands how an endpoint detection and response (EDR)

system logs and analyzes process behavior can adjust their payload execution to avoid common triggers. A red teamer who knows what a cloud team's monitoring stack looks like can shape their actions to test the blind spots that actually exist.

Defensive knowledge also helps a red team communicate its findings. A red teamer skilled only in offensive techniques might report an attack path to domain admin. One who knows both offense and defense can explain why the attack path existed, how it should have been detected, and what to change so it doesn't work again.

Building this kind of understanding takes time and effort. Read extensively about detection engineering. Talk to the SOC and the incident response teams about what they see and how they handle it. Study the configurations and tools in your environment, even when an engagement doesn't strictly require it. Red teams should know defense as deeply as offense.

Principle 9: Keep Trying

Red teaming is full of setbacks. Payloads get flagged, credentials don't work, access gets cut off, and you'll spend hours troubleshooting a problem only to find a missing slash in a URL. These moments can be demoralizing, but they're also where growth happens.

Persistence is one of the most underrated traits in a red teamer. The trick is to stay engaged and curious, working on the problem even when things don't go your way. Persistence and brute force are different skills, though, and pushing blindly into a failing approach burns time and attention. The skill is knowing when to step back, treat a failed attack path as a useful data point, and try a different angle. What did failure tell you? What's a different approach that could work?

Persistence also includes taking care of yourself. This work is mentally and emotionally demanding, and it's easy to internalize the pressure to solve every problem on your own. Burnout doesn't produce better results. The operators who sustain long careers in this field pace themselves and recognize rest as part of the process. When you feel stuck, talk to a teammate, take a break, or try something completely different. Some of the best breakthroughs come immediately after the point where you almost gave up.

Red teaming rewards consistency. Keep showing up and learning, and you'll build the skills and character that sustain a long career in this field.

Principle 10: Never Stop Learning

Red teaming changes constantly. Today's latest technologies will become tomorrow's legacy systems. Modern defensive methods and controls replace outdated approaches. Threat actors continually invent new tactics and techniques. The practitioners who stay relevant are the ones who treat learning as part of the job.

There are many ways to keep growing. Hands-on platforms like Hack The Box and TryHackMe provide lab environments in which to practice techniques and sharpen your technical skills. Threat intelligence reports and offensive research blogs can help you track how attack tradecraft is

changing. Conferences like DEF CON, Black Hat, and BSides build community and reveal emerging trends. Home labs let you experiment safely with technology you fully control. By reverse engineering malware and studying detection rules, you can build fluency in both offensive and defensive techniques.

The most lasting growth happens when we teach others. Writing a blog post, speaking at a meetup, mentoring a junior teammate, or contributing to open source communities forces a deeper kind of understanding than passive consumption ever does. The security community has always been shaped by people who were willing to share what they learned.

The best red teamers stay humble and curious. Even years into their careers, they approach new problems with a willingness to ask questions and learn from people more experienced than they are. The work itself is interesting enough to sustain that kind of engagement over a long career.

Summary

Red teaming is first and foremost a mindset and a mission. In this chapter, we examined how it compares with other forms of security testing, emphasizing its objective-driven, full-scope, and context-aware nature. Through real-world examples and game theory principles like zero-sum dynamics, information asymmetry, and bounded rationality, we framed red teaming as a strategic tool that helps organizations shift from reactive to proactive defense postures. We also outlined the mindset required to operate effectively in this role, discussing the need to challenge assumptions, embrace ambiguity, and prioritize integrity.

As we move into the next chapters, we'll build on this foundation and explore how to design, lead, and scale effective red team programs that deliver lasting rewards across an organization.